사용자가 입력한 미세먼지(pm10)와 오존 데이터를 바탕으로 평균을 계산하고, 이를 기준으로 대기질 등급을 판단하는 프로그램을 어떻게 구현할 수 있을까?

사고 단계 >

사용자가 입력한 데이터를 바탕으로 평균을 구하고 대기질 등급을 판정하려면, 파이썬의 **내장 함수(sum, len)**와 **if-elif-else 조건문**을 결합하는 방식이 가장 직관적이고 효율적입니다.

입력받은 수치들을 리스트에 저장해 두었다고 가정할 때, 다음과 같은 흐름으로 구현해 볼 수 있습니다.

💡 핵심 구현 로직 추천

1. **평균 계산 (sum과 len 활용):**

리스트에 누적된 데이터의 총합을 구하는 sum() 함수와 데이터의 개수를 구하는 len() 함수를 나누어 평균값을 계산합니다.

2. **등급 판정 (if-elif-else와 논리 연산자 활용):**

계산된 평균값을 기준으로 if-elif-else 다중 조건문을 세웁니다. 미세먼지와 오존 두 가지 조건을 모두 만족해야 할 때는 and 연산자를, 하나라도 기준을 초과하면 나쁜 등급을 줄 때는 or 연산자를 활용하면 좋습니다.

💻 추천 코드 뼈대

python

예시: 사용자가 입력하여 리스트에 저장된 데이터

Gemini에게 물어보세요

Pro

추천 코드 뼈대

python

```python
# 예시: 사용자가 입력하여 리스트에 저장된 데이터
pm_list = [20.0, 50.0, 100.0]  # 미세먼지(PM10) 데이터
o3_list = [0.02, 0.05, 0.08]   # 오존(O3) 데이터

# 1. sum()과 len()을 활용한 평균값 계산
avg_pm = sum(pm_list) / len(pm_list)
avg_o3 = sum(o3_list) / len(o3_list)

print(f"평균 PM10: {avg_pm:.2f}, 평균 O3: {avg_o3:.3f}")

# 2. if-elif-else 구문을 활용한 대기질 등급 판정
# 미세먼지와 오존 모두 기준치 이하일 때 '좋음', 둘 중 하나라도 높으면 '보통' 또는 '나쁨'
if avg_pm <= 30.0 and avg_o3 <= 0.030:
    grade = "좋음 🟢 (야외 활동하기 좋습니다!)"
elif avg_pm <= 80.0 or avg_o3 <= 0.090:
    grade = "보통 🟡 (무난한 대기질입니다.)"
else:
    grade = "나쁨 🔴 (마스크 착용을 권장합니다!)"

print(f"종합 대기질 등급: {grade}")
```

이 방식을 사용하면 코드가 매우 간결해지고, 나중에 판단 기준(수치)이 바뀌더라도 조건문 부분만 수정하면 되기 때문에 유지보수하기도 아주 좋습니다. 이 로직을 'Air Guard' 시스템에 적용해 보세요!

소스

입력데이터 중 오류 값(음수,비정상적으로 큰 값)을 어떻게 처리해야하고 프로그램의 안정성을 높일 수 있을까?

사고 단계 >

사용자님이 입력받는 데이터(미세먼지, 오존)는 현실 세계의 물리적인 측정값이기 때문에, 논리적으로 불가능한 수치가 입력되었을 때 이를 걸러내는 예외 처리 로직이 필수적입니다.

파이썬의 반복문 제어 명령어인 **continue**와 **break**를 활용하면 이러한 비정상 데이터를 아주 효과적으로 차단하여 프로그램의 안정성을 크게 높일 수 있습니다. 다음과 같이 적용해 보는 것을 추천합니다.

🛡️ 비정상 데이터 처리 및 반복 구조 개선 방법

1. continue 활용: 잘못된 입력 무시하고 다음으로 넘어가기

- **상황:** 대기질 농도는 물리적으로 0보다 작을 수 없습니다. 사용자가 실수로 음수(예: -10)를 입력할 수 있습니다.
- **처리 로직:** 입력값이 음수일 경우, 이를 잘못된 입력(단순 오타 등)으로 간주합니다. 이때 continue를 사용하면 해당 측정값을 리스트에 저장하지 않고, 아래 코드를 건너뛴 채 곧바로 다음 회차의 입력(다음 측정)으로 넘어갈 수 있습니다.

2. break 활용: 치명적인 오류 감지 시 프로그램 즉시 중단

- **상황:** 미세먼지 수치로 2000 ㎍/㎥ 이상 등 현실적으로 지구상에서 측정되기 불가능한 값이 들어올 수 있습니다.
- **처리 로직:** 이렇게 터무니없이 큰 값은 단순 오타가 아니라 **측정 센서의 심각한 고장이나 시스템 오류**로

Gemini에게 물어보세요

Pro

- **처리 로직:** 이렇게 터무니없이 큰 값은 단순 오타가 아니라 **측정 센서의 심각한 고장이나 시스템 오류**로 간주해야 합니다. 이때는 무의미한 입력을 계속 받는 것보다 break를 사용하여 전체 프로그램(반복문)을 즉시 멈추고 관리자에게 오류 상황을 알리는 것이 안전합니다.

💻 적용 코드 예시

python

```python
# for 반복문 내부에서 데이터 입력 후 즉시 검증하는 로직
for i in range(day_count):
    pm = float(input(f"{i+1}일차 미세먼지(PM10) 입력: "))
    o3 = float(input(f"{i+1}일차 오존(O3) 입력: "))

    # 1. 음수 입력 시 (continue 활용)
    if pm < 0 or o3 < 0:
        print("경고: 대기질 수치는 음수가 될 수 없습니다. 해당 입력은 무시됩니다.")
        continue  # 리스트에 추가하지 않고 다음 순서(i+1)로 바로 넘어감

    # 2. 비정상적으로 큰 값 입력 시 (break 활용)
    if pm >= 2000 or o3 >= 1:
        print("긴급 오류: 센서 고장이 의심되는 치명적인 수치입니다. 시스템을 정지합니다.")
        break  # 반복문을 즉시 완전히 탈출하여 프로그램 중단

    # 3. 정상 데이터인 경우에만 리스트에 저장
    pm_list.append(pm)
    o3_list.append(o3)
```

이렇게 구성하시면 평가 기준인 break/continue 활용 조건을 완벽하게 충족하면서도, 실제 상용 프로그램처럼 튼튼하고 안정적인 예외 처리 구조를 가지게 됩니다!

소스